

Empirical Evaluation of Dynamic Augmented Pooled Testing

June 3, 2026

1 Empirical Evaluation

We now evaluate the augmented dynamic pooled testing algorithms implemented in the current codebase. The experimental organization follows the structure used in Lopez’s *Dynamic, Welfare-Maximizing Pooled Testing*: we first study the same small problem instances for which exact dynamic optimization is feasible, and then extend the comparison to the augmented setting.

The central distinction in the present setting is the test signal. In the classical welfare-maximizing model, a pooled test returns a binary positive or negative result. In the augmented model, a test on pool t returns the exact number of infected agents in the pool,

$$r(t, Z) = |t \cap Z|.$$

Thus, all classical information remains available, but positive tests are further refined by their infection count. This allows a dynamic policy to condition future tests on a richer history.

All empirical routines used for this section are collected in `augmented/notebooks/nick_empirical_replication_augmented.ipynb`. The notebook regenerates the tables in this section from synthetic instances matching the parameter regimes used in Lopez’s thesis: small instances with $(N, G, B) = (3, 3, 2)$ and $(5, 5, 3)$.

2 Algorithms Evaluated

For small instances, we compare the following assignment rules.

- **Non-pooled testing.** The testing authority tests the B agents with largest individual expected welfare.
- **Static non-overlapping testing.** The authority chooses up to B disjoint pools before seeing any outcomes.
- **Static overlapping testing.** The authority chooses up to B pools before seeing any outcomes, allowing agents to appear in multiple pools.
- **Classical optimal dynamic testing.** The authority chooses pools sequentially, observing only whether each pool is positive or negative.
- **Augmented optimal dynamic testing.** The authority chooses pools sequentially, observing the exact infected count in each pool.

- **Augmented greedy dynamic testing.** The authority repeatedly chooses the pool maximizing immediate expected clearing welfare under the current posterior marginal infection probabilities.
- **Counting-based augmented greedy testing.** The same myopic objective is used, but posterior marginals are computed from the full history by enumerating all infection profiles consistent with the observed count outcomes.

The resulting benchmark chain is

$$U^{\text{single}} \leq U_{NO}^s \leq U_O^s \leq U^D \leq U_A^D \leq U^{\text{max}}.$$

The first three terms are static or non-adaptive benchmarks. The term U^D denotes the optimal dynamic value under binary classical tests, while U_A^D denotes the optimal dynamic value under augmented count tests. The final term is the utility that would be obtained if every healthy agent could be certified.

3 Relation to Lopez’s Algorithms

The augmented codebase keeps the main structure of Lopez’s thesis, but changes the test outcome. Lopez studies binary pooled tests: a pool is either negative or positive. The augmented code studies count tests: a pool returns the exact number of infected agents. This changes the information available after a positive test.

The first three rows are essentially inherited benchmarks. They are necessary because they make the augmented results comparable to the original welfare testing framework. The new part starts with the dynamic algorithms. In the classical model, a positive test only says that at least one agent in the pool is infected. In the augmented model, the same test can say that exactly one, two, or more agents are infected. The policy therefore has a richer history, and the optimal dynamic tree has more branches.

The most important new comparison is U^D versus U_A^D . Both values use the same population, utilities, budget, and pool-size limit. The only difference is the information returned by a positive test. This makes the comparison clean: any improvement from U^D to U_A^D is the value of the count signal.

Several exploratory proposals were added in the augmented codebase. The first is full-history posterior counting, which recomputes posterior marginals by enumerating all worlds consistent with the observed count history. The second is Gibbs sampling with swap moves, which approximates the same posterior when enumeration is too expensive. The third is the independence-gap experiment, which directly tests when the greedy approximation based on marginal probabilities is misleading. The fourth is the super-node or VW idea, which tries to compress groups of previously tested agents into larger units. Current experiments suggest that the simplest VW version rewrites the myopic greedy score rather than improving it, so its theoretical value remains open.

4 Algorithmic Descriptions

4.1 Optimal Dynamic Augmented Testing

The optimal augmented dynamic algorithm operates over an information state (k, R, C) . Here, k is the number of tests already used, R is the set of infection profiles still consistent with the observed history, and C is the set of agents already certified healthy. For each candidate pool t ,

Lopez algorithm	Augmented counterpart	Main change
Non-pooled testing	<code>u_single</code>	Same benchmark. Test the best individual agents by expected value.
Static non-overlapping testing	<code>solve_static_non_overlapping</code>	Same static idea. Count outcomes do not help because all pools are fixed before results are observed.
Static overlapping testing	<code>solve_static_overlapping</code>	Same static idea, allowing repeated agents across pools. Augmented outcomes still do not affect the assignment because there is no adaptation.
Optimal dynamic testing	<code>solve_optimal_dapts</code>	This is the main augmented version. The dynamic tree branches on $r \in \{0, \dots, t \}$ instead of only positive/negative.
Greedy dynamic testing	<code>greedy_myopic_expected_utility</code>	Same myopic principle: pick the pool with largest immediate clearing value. The future posterior can now use count information.
Posterior updates	<code>bayesian_update_by_counting</code> <code>gibbs_update</code>	The posterior must satisfy exact count constraints. This makes the dependence between agents more visible than in the binary setting.
Joint-distribution warning	<code>independence_gap.py</code>	The augmented project measures the error from multiplying posterior marginals when agents are actually correlated after tests.

Table 1: Connection between the algorithms in Lopez’s thesis and the current augmented implementation.

the algorithm partitions R by the count outcome $r(t, Z)$ and recurses on each resulting branch. If $r = 0$, all agents in t are added to C .

Algorithm 1: Optimal Dynamic Augmented Testing

1. Initialize R to all 2^N infection profiles and $C = \emptyset$.
2. If $k = B$, return the posterior mass of R multiplied by the utility of agents in C .
3. For each pool t with $|t| \leq G$, partition R into sets $R_r(t) = \{Z \in R : |t \cap Z| = r\}$.
4. For each outcome r , recurse on $(k+1, R_r(t), C \cup t)$ if $r = 0$, and on $(k+1, R_r(t), C)$ otherwise.
5. Select the pool maximizing the sum of branch values.
6. Reconstruct the policy tree from the maximizing choices.

This algorithm is implemented in `augmented/solver.py`. It is exact, but its state space is exponential in the number of infection profiles. It is therefore used only for small instances and as a benchmark for heuristic methods.

4.2 Classical Dynamic Testing

The classical dynamic benchmark uses the same information-state recursion, but each tested pool has only two possible outcomes: negative, when $t \cap Z$ is empty, and positive otherwise. This

algorithm is implemented in `augmented/classical_solver.py`. Comparing U^D and U_A^D isolates the value of the augmented count signal.

4.3 Greedy Dynamic Augmented Testing

The greedy dynamic algorithm selects a single test at each step using the current posterior marginal infection probabilities. If $\tilde{p}_i$ denotes the posterior probability that agent i is infected, the immediate clearing score of a pool t is

$$\left(\prod_{i \in t} (1 - \tilde{p}_i) \right) \left(\sum_{i \in t} u_i \right).$$

After observing the count result, posterior probabilities are updated and the process repeats until the budget is exhausted.

Algorithm 2: Greedy Dynamic Augmented Testing

1. Initialize posterior infection probabilities to the prior vector.
2. At each remaining budget step, choose the pool maximizing immediate expected clearing welfare.
3. Observe the augmented count outcome $r = |t \cap Z|$.
4. If $r = 0$, mark every agent in t as certified healthy.
5. Update posterior infection probabilities using the observed history.
6. Repeat until no tests remain.

This procedure is implemented in `augmented/greedy.py`. The sequential version uses one-test posterior updates, while the counting version recomputes posterior marginals from the entire history.

4.4 Posterior Inference by Counting and Gibbs Sampling

The exact posterior after a history h is a distribution over full infection profiles,

$$\Pr[Z \mid h].$$

For small N , this distribution can be computed by enumerating all 2^N profiles and retaining only those consistent with every observed count. For larger N , enumeration is infeasible. The code therefore also implements a Gibbs sampler with Metropolis-Hastings swap moves in `augmented/bayesian.py`.

The purpose of the sampler is not to directly optimize a testing plan. Rather, it approximates posterior marginal infection probabilities $\Pr[Z_i = 1 \mid h]$, which can then be used by scalable greedy policies. The swap moves are included because exact-count constraints can make single-coordinate Gibbs updates become stuck: when a pool is known to contain exactly one infected agent, moving between feasible assignments often requires simultaneously turning one infected agent healthy and one healthy agent infected.

5 Small Instances: Exact Benchmarks

Following Lopez’s small-instance experiments, we evaluate synthetic populations with $(N, G, B) = (3, 3, 2)$ and $(N, G, B) = (5, 5, 3)$. For each instance, health probabilities and utilities are sampled independently from the uniform distribution on $[0, 1]$. Because the local augmented implementation represents p_i as infection probability, the notebook samples health probability q_i and passes $p_i = 1 - q_i$ to the solvers.

Constraints	Algorithm	Average Expected Welfare	Standard Error
$(N, G, B) = (3, 3, 2)$	Non-pooled	generated by notebook	generated by notebook
	Static non-overlapping	generated by notebook	generated by notebook
	Static overlapping	generated by notebook	generated by notebook
	Classical dynamic optimal	generated by notebook	generated by notebook
	Augmented greedy dynamic	generated by notebook	generated by notebook
	Augmented counting greedy	generated by notebook	generated by notebook
	Augmented dynamic optimal	generated by notebook	generated by notebook
$(N, G, B) = (5, 5, 3)$	Non-pooled	generated by notebook	generated by notebook
	Static non-overlapping	generated by notebook	generated by notebook
	Static overlapping	generated by notebook	generated by notebook
	Classical dynamic optimal	generated by notebook	generated by notebook
	Augmented greedy dynamic	generated by notebook	generated by notebook
	Augmented counting greedy	generated by notebook	generated by notebook
	Augmented dynamic optimal	generated by notebook	generated by notebook

Table 2: Average expected welfare across synthetic small instances. The notebook regenerates this table and can be configured to use either a quick sample size or the 1,000 instances used by Lopez.

These experiments serve two purposes. First, they validate the augmented implementation against exact optimal values. Second, they separate the benefit of dynamic assignment from the benefit of augmented count information. The classical dynamic value U^D uses the same testing budget and pool-size constraints as the augmented dynamic value U_A^D , but observes only binary outcomes.

6 Augmented Versus Classical Dynamic Testing

The difference $U_A^D - U^D$ measures the value of replacing binary test outcomes with exact count outcomes while keeping the rest of the dynamic model fixed. When a pool tests negative, both models obtain the same immediate welfare: all agents in the pool are certified healthy. The additional value of the augmented model arises after positive tests. A classical positive result only states that at least one member of the pool is infected. An augmented positive result distinguishes between one, two, or more infections, which refines the set of infection profiles consistent with the history and can improve future pool selection.

This effect is most visible when the testing budget is large enough for the policy to use information from earlier tests, but still small enough that not all agents can be individually resolved. In such regimes, the augmented signal can improve second- and later-stage decisions even when it provides no immediate clearing utility at the moment it is observed.

7 Greedy Posterior Updates and the Independence Gap

The augmented greedy algorithm is computationally attractive because it uses posterior marginal probabilities rather than the full joint posterior. This is also its main conceptual limitation. After overlapping tests, agents' infection states are generally dependent. Thus, the probability that an entire pool is healthy is not necessarily the product of the agents' marginal health probabilities.

For example, suppose a prior symmetric test on agents $\{1, 2\}$ returns count 1. Then each agent may have posterior infection probability $1/2$, but the probability that both are healthy is zero. A product-of-marginals approximation would incorrectly assign probability $1/4$ to this event. The module `augmented/independence_gap.py` measures this discrepancy by comparing the true posterior count distribution of a candidate pool against the Poisson-binomial distribution implied by posterior marginals.

This issue is directly analogous to the limitation discussed by Lopez for classical greedy dynamic testing: greedy policies obtain scalability by ignoring some joint dependencies induced by earlier tests. The augmented setting makes the dependency structure richer because intermediate count outcomes can create strong negative correlations among agents inside tested pools.

8 Scope of the Replication

This first artifact focuses on the part of Lopez's empirical evaluation that can be replicated exactly with the current augmented code: small instances where all relevant policies can be evaluated by enumeration. This is the cleanest place to introduce the augmented model because the classical and augmented dynamic values can be computed on the same populations.

The large-instance experiments from Lopez use $N = 50$ agents, utilities drawn from $\{1, 2, 3\}$, health probabilities drawn uniformly from $[0, 1]$, and pool-size constraints $G \in \{3, 5\}$ with varying budget B . We do not include those experiments here yet. Exact augmented dynamic optimization is infeasible at that scale because the profile space contains 2^{50} possible infection states. A later version of this section should compare scalable augmented heuristics against the large-instance baselines, but that is intentionally left outside the present draft.

The present notebook therefore has a narrow purpose: reproduce the small-instance empirical tables, add augmented dynamic and augmented greedy columns, and measure the gap between classical binary outcomes and augmented count outcomes.

9 Discussion

The empirical structure mirrors the original dynamic welfare-maximizing pooled testing evaluation, but the augmented model changes the interpretation of adaptivity. In the classical setting, dynamic assignment improves welfare by conditioning future pools on binary positive and negative outcomes. In the augmented setting, dynamic assignment also conditions on the magnitude of positive outcomes. This additional information never decreases the optimal dynamic value and may strictly improve it.

The small-instance experiments are therefore the appropriate validation point: they allow direct computation of U^D and U_A^D on the same populations. They also show the practical challenge for the next phase. Exact augmented optimization is not scalable, and greedy policies rely on marginal posterior approximations that can be inaccurate when tests overlap. The main research question suggested by these experiments is whether one can keep enough of the joint posterior structure to improve greedy decisions without returning to exponential enumeration.